ISTA 130

`for` Loops

What's new?

More work with functions

Adding `for` loops (repetition)

NOTE: Don't cut-and-paste stuff from PDFs into scripts!

Last time

More turtle

More functions

We drew a house with functions!

Types

Types are categories of objects that share common properties and behaviors. Like integers. And turtles.

The built-in `type()` function can tell us what kind of object we're looking at:

```
>>> import turtle
>>> tallulah = turtle.Turtle()
>>> type(tallulah)
 <class 'turtle.Turtle'>
>>>
>>> type(33)
 <class 'int'>
>>>
>>> type('hmmmm')
 <class 'str'>
>>>
>>> type(type)
 <class 'type'>
```

Types of Values

Python recognizes many different *types* of values:

- `int` (short for integer)
  - e.g., `0, 7, -21, 4593211, -101`
  - Do not put commas in ints (`1000`, not 1,000)
- `float`
  - e.g., `3.14159, 2.71828, 5.0, -0.99999`
- `str` (short for string)
  - e.g., `"blue", 'salmon', "why is that salmon blue?"`

Python calculates

Let's do some.

Order of operations: PEMDAS

1. Parentheses: `x * (x + z)`
2. Exponentiation: `x ** y`
3. Multiplication and Division: `x * x, x / y`
4. Addition and Subtraction: `x + x, x - y`

Otherwise left to right, except exponentiation.

You can use parentheses to force the order you want (or just to make your code more readable).

Functions

Our `main()` function is our algorithm - a series of step-by-step instructions that solves a problem or performs a task.

We would like our main function to be readable - reasonably short with instructions that have informative names.

We accomplish this goal with functions.

```
docstr
import modules
def functions
```



`main()`



Functions

Functions implement sub-algorithms that help our main algorithm reach its goal. They solve a small problem or accomplish a small task that helps the main algorithm do its job.

Functions are appropriate for tasks that are repeated more than once and for logically related tasks such as reading in input.

Don't do too much in any given function.

Functions

`main()` is a function that is called at the very end of your script. It starts executing instructions as soon as we start the program (i.e. when you run `python script.py`)

Functions defined before `main()` (in the function definitions area) only execute if and when called. `square()` is a short algorithm that only runs when called.

```
if __name__ == '__main__':
    main()
```

Functions

When you call a function, you must give it any information it needs to do its job:

➔ This won't work: `tallulah.forward()`
➔ This will: `tallulah.forward(100)`

What info does `square(??)` need to do its job?

➔ A turtle (any turtle will do)
➔ A size (any size will do)

Function definition vs function call

	Function Definition	Function call
Syntax	`def fn_name(params):` `    fn_body`	`fn_name(args)`
Purpose	Store code for later use as a one-line instruction	Execute the code in the function body with a one-line instruction.
Executes	When called by other code	When encountered
Parameters vs Arguments	Parameters are variables that take on the values of the corresponding arguments in the function call during execution.	Arguments can be variables or literals (explicit values). Gets info into the function for added flexibility.

Does `square()` need to know how `main()` names its turtles?

No, `square()` just needs to know what turtle to use.

`square()` never knows what names `main()` uses for its turtles.

`main()` never knows what names `square()` uses for its turtles.

Scope

The region of code where a variable is visible (i.e., where Python knows what you're talking about).

- ➔ Parameters are only visible within their functions.
- ➔ Variables defined within functions are only visible to Python within those functions.
  - ◆ `main()` 's variables are visible within `main()` only.
  - ◆ `square()` 's variables are visible within `square()` only.

Modifying Code

Making changes in several places is an error-prone process, not to mention tedious.

We can solve this by using a variable.

To create a variable, we just need to choose a name and assign a value to it, like this:

➔ `size = 100`
➔ Read this: "`size` gets assigned `100`"
➔ Afterwards, we say: "`size` refers to `100`" or "`size` points at `100`".

```
size = 100
```

Think of size as a bucket. We just put a `100` object into it. It will stay there until we replace it with some other object:

➔ `size = 65536`

Now `size` refers to `65536`.

These types of instructions are called ***assignment statements***.

Variables

If we wanted to make our houses bigger, for example, we could just change one line, so:

➔ `size = 200`

Now

```
tallulah.goto(x + size * 1.25, y)
```

still puts our house where we want it without us having to change anything.

Terminology Summary

Variable: a name that refers to an object.

➔ `size = 200`

➔ `tallulah = turtle.Turtle()`

Parameter: variable used by a function to refer to argument values passed when the function is called.

➔ `def square(a_turtle, side_len):`

Argument: an object passed to a function. Can be a variable or a literal.

➔ `square(tallulah, 75)`

➔ `square(myrtle, 200)`

Python max line length is 80

By convention.

Don't let your comment lines wrap around.

The `print` Function

Prints to the console.

Takes zero or more arguments.

Puts a space between each argument (essentially - replace each comma with a space).

`print()` with no arguments prints a blank line.

Let's try it.

Some Squares

Think about drawing a diagonal line composed of squares.

What we want to do is very simple, but the code is very long.

What if we want a line of 100 squares? We will have to cut and paste 100 times! Making code hundreds of lines long!

Repeating Code

Many times we'll want to tell Python to do something and then repeat it over and over.

- ➔ The programming term for this is ***iteration*** (or ***looping***).
- ➔ We need a way to tell Python what code to repeat and how many times to repeat it.
- ➔ There are two types of loops in Python. Today we examine **`for` loops**.

`for` Loops

The `for` loop in Python looks like this:

```
for i in range(10):
    statement1
    statement2
    ...
```

A Python ***statement*** is a Python instruction.

`for` Loops

```
for i in range(10):
    statement1
    statement2
    ...
```

We'll analyze loops in much more detail, but for now all you need to know to use a for loop is:

- ➔ Change the `10` in `for i in range(10):` to the number of times you want something repeated.
- ➔ Place the code you want repeated under this and indent it one level (4 spaces).

`for` Loops

Like writing functions, using loops improves our code:

- ➔ More clear and concise
- ➔ Easier to maintain
  - ◆ When we make a change we don't have to remember to change lots of other parts of our code.
- ➔ We don't have to type as much - code reuse.

A New `square()`: So Pretty!

```
def square(a_turtle, side_len):
    for i in range(4):
        a_turtle.forward(side_len)
        a_turtle.right(90)
    return None
```

Notice the indentation:

- ➔ the `for` loop is indented one level to put it inside the function.
- ➔ The turtle method calls are indented two levels to put them inside the `for` loop.
- ➔ `return None` is only indented once, because it's inside the function but not inside the for loop.

In Python, a colon ending a line always needs to be followed by indented statement(s)

```
for i in range(4):
```

`i` is a variable called the ***loop control variable***.

This statement turns it into a counter - it steps from `0` to `3`: `0, 1, 2, 3`

In Python, we often stop one number short, which we call right-side exclusive.

`for` Loops

We can use a variable to tell our for loop how many times to repeat:

```
for i in range(num_reps):
```

`for` Loops: Example

```
for i in range(10):
    square(a_turtle, 20)
    a_turtle.penup()
    a_turtle.goto(x + 10, y + 10)
    a_turtle.pendown()
```

We're telling Python:

- ➔ "Draw a square and then move the turtle."
- ➔ "Repeat this 10 times"